

NIRBENCH-DL v0.1

Methods, results and user guide

Dário Passos

02/10/2026

DeepLight Laboratory, Physics Department, University of Algarve, Portugal

1 Purpose and datasets

NIRBENCH-DL is a research tool for comparing deep-learning architectures across near-infrared (NIR) chemometric regression tasks. It gives researchers fixed datasets, a common evaluation procedure, and an interface for testing new models. The long-term aim is to identify architectural choices that work well across different spectral prediction problems, i.e., which type of layers of modules generalize for spectra analysis.

Version 0.1 compares 13 published neural-network architectures designed for spectral analysis with optimized partial least squares (PLS) and the pretrained TabPFN-3.5 regressor on 30 tasks. Neural-network settings/hyperparameters are fixed in the benchmark; only the training epoch budget is selected per model and task. For PLS we perform a joint preprocessing and latent-variable optimization, the standard procedure in NIR Chemometrics. This is therefore a comparison of out-of-the-box DL models + TabPFN-3.5 (no HPO done) against optimized PLS (considered the golden standard in chemometrics).

Task	Dataset / target	Train	Test	Channels
1	Wheat kernels protein	415	108	100
2	Wheat flours protein	112	28	525
3	Tecator moisture	172	43	100
4	CGL NIR grain glucose	153	78	117
5	Cucurbitaceae fruit ssc	200	100	229
6	NIR tomato1 ssc	126	42	174
7	NIR tomato2 ssc	135	45	174
8	NIR tomato3 ssc	80	26	174
9	NIR tomato4 ssc	81	27	174
10	NIR tomato5 ssc	66	22	174
11	Olive oils c16	149	38	612
12	NIR DieselFuels cn	298	83	401
13	NIR milk protein	963	261	256
14	NIR Pharmaceutical Tablets assay	615	40	650
15	CEOT pear 2021 brix	3,204	804	874
16	CEOT pears 2019 brix	2,640	660	875
17	Barley sensAIfood Perten moisture	120	30	141
18	Barley sensAIfood Perten protein	120	30	141
19	Wheat sensAIfood Grainit moisture	318	80	351
20	Wheat sensAIfood Grainit protein	323	80	351
21	CEOT pear 2021 snv brix	3,204	804	459
22	Mango S1 dm	3,132	782	101
23	Mango S2 dm	1,086	277	101
24	Mango S3 dm	3,964	1,002	101
25	Mango S4 dm	1,158	290	101
26	Corn sensAIfood Perten moisture	120	30	141
27	Corn sensAIfood Perten protein	120	30	141
28	Wheat sensAIfood Perten moisture	120	30	141
29	Wheat sensAIfood Perten protein	120	30	141
30	Wheat sensAIfood CRAW protein	4,036	1,010	700

Table 1: Fixed partitions after loading. Task numbers match the dataset-folder prefixes and the result tables. SSC: soluble solids; dm: dry matter; cn: cetane number; c16: C16 fatty acid. Tasks 22–25 use the original Anderson mango dataset (V2), separated by season. The partitions keep every equal-DM group within a season together and allocate 80% of groups to training and 20% to test with seed 42. There is no explicit fruit ID, so identical DM is a conservative proxy for repeated measurements; coincidentally equal values are kept together as well. The spectrum counts differ slightly from an exact 80/20 split.

2 Evaluation methodology

2.1 Fixed splits and preprocessing

Each task has stored training and test CSV files in `datasets/`. The shared loader, `src/data_loading.py`, removes unnamed/index columns and a leading ID-like column identified by its nonnumeric heading, aligns spectral columns, uses the last column as the target, and removes rows containing missing values. Table 1 reports the resulting sizes. Dataset origins and source links are listed in Section 7.

For DL models, each wavelength and the target are standardized using means and standard deviations fitted only to the current training rows. During cross-validation (CV), scalers are fitted separately inside each fold; during final training, they are refitted on the full training partition. Predictions are inverse-transformed before scoring. CNNs receive a channel dimension, giving inputs of shape `(samples, wavelengths, 1)`. Existing preprocessing in a supplied dataset, such as task 21’s SNV spectra, is retained. No test labels are used to choose preprocessing, epochs, or model settings.

2.2 Neural networks: choose epochs, then evaluate ten fresh fits

For each model–task pair:

1. Split the stored training partition into five shuffled folds with seed 12345. Fit a fresh model on four folds and validate on the fifth.
2. Train for up to 600 epochs, with early stopping after 100 epochs without validation-loss improvement. Record the epoch with the lowest validation loss in each fold.
3. Use the median of the five best epochs as the final training budget. Discard all fold models.
4. Fit ten fresh models on the full training partition for exactly that budget, using seeds 12345–12354. These fits have no validation split or early stopping; applicable learning-rate schedules remain active.
5. Evaluate every fit on the same test partition and report the mean and sample standard deviation of test RMSE and R^2 .

All 13 implementations use MSE loss and Adam in the current runner. Initial learning rates, decay schedules, regularization, and batch sizes are model-specific (Table 2). Five-fold CV selects epochs; it does not search learning rates or architectures.

2.3 PLS: jointly choose preprocessing and latent variables

Five-fold CV with shuffled folds and seed 12345 searches nine preprocessing choices: raw spectra, standard normal variate (SNV), multiplicative scatter correction (MSC), first and second Savitzky–Golay derivatives, and SNV or MSC followed by either derivative. Derivatives use a nine-channel window and polynomial order two. For each choice, the search considers 1–20 latent variables, limited by fold size and spectral width.

Selection minimizes pooled out-of-fold RMSE in the original target units. SNV operates on each spectrum independently; MSC references and `PLSRegression(scale=True)` scaling are estimated from each fold’s training rows. Ties are resolved by the fixed preprocessing order and then fewer latent variables. The selected pipeline is fitted once on the full training partition and evaluated on the test partition. “Optimized PLS” refers to this specified search space. For mango, the outer train/test split separates inferred fruit groups; the internal CV remains row-based and can share repeated measurements across folds, so its validation scores may be optimistic.

2.4 TabPFN-3.5: pretrained regression

The standard TabPFN-3.5 checkpoint is evaluated through `TabPFNRegressor`, with package version 9.0.0 in the reported run. The complete standardized training partition provides context to the pretrained model. There is no epoch selection or CV. Ten seeds (12345–12354) vary the internal ensemble, with automatic ensemble size and `fit_preprocessors` mode. Test predictions are returned to original units; training metrics are omitted because the training labels are already in the model’s context. Checkpoint identity and inference settings are recorded with the results.

3 Architectures and implementation choices

The neural networks are implemented in TensorFlow/Keras under `models/`, with a single continuous output. They are local implementations and interpretations of the cited architectures. Where papers leave details unspecified, these versions may differ from the original authors’ code. The results evaluate these implementations and their benchmark settings.

Model	Main architectural choice	Training setting in this benchmark
DeepSpectra	Inception-style 1D convolutions with dropout and L2 regularization.	Adam, initial LR 0.00438; inverse-time decay.
IPA	Compact Inception CNN with parallel convolutional branches.	Adam, initial LR 10^{-3} ; exponential decay.
1D Inception-ResNet	Residual Inception blocks and a regression head.	Adam, initial LR 10^{-3} ; plateau reduction.
MarkSpectra	A dense network using selected wavelength “marks.”	Adam, initial LR 10^{-2} ; inverse-time decay.
ResidualSpectra	Residual 1D CNN adapted to regression.	Adam, LR 10^{-3} .
SCNet	Multiscale convolutions with shortcut feature concatenation and a dense head.	Adam, LR 10^{-3} .
Spectraformer	Convolutional features with Transformer blocks.	Adam, LR 10^{-4} .
SpectraTr	Spectral patches processed by a Transformer; patching adapts to input length.	Adam, initial LR 10^{-4} ; plateau reduction.
SpectraNet32	Residual CNN with parametric GELU, dropout and L2 regularization.	Adam, initial LR 10^{-3} ; exponential decay; clip norm 0.8; batch 128.
SpectraNet53	Deep CNN with PReLU/GELU, dropout and L2 regularization.	Adam, initial LR 10^{-4} ; exponential decay; clip norm 0.5.
1D-CNN_1L_3	One wide convolution (kernel 31), then dense layers 128–64–16–1.	Adam, LR 10^{-3} ; L2 10^{-3} .
1D-CNN_1N_3	One narrow convolution (kernel 5), with the same dense head.	Adam, LR 10^{-3} ; L2 10^{-3} .
1D-CNN_3_1	Three convolution blocks (16/32/64 filters; kernels 21/11/5), pooling and a dense head.	Adam, LR 10^{-3} ; L2 10^{-3} ; dropout 0.20.

Table 2: The 13 DL architectures. All use MSE loss; batch size is 16 unless stated otherwise. Model names link to their source papers; the final three are simple benchmark baselines. LR denotes learning rate.

MarkSpectra ranks wavelength marks by univariate OLS R^2 and keeps approximately 30%, with a minimum of ten. Selection is repeated using only each fold’s training data, then refitted on the full training partition for final evaluation. It must not be computed from the test labels.

For 1D Inception-ResNet, plateau reduction monitors validation loss during CV and training loss during final fits. SpectraTr monitors training loss in both phases. The input dimension follows each dataset, so model capacity can change with spectral width. Random seeds are controlled, but numerical results need not be identical across software versions or hardware.

The reference methods use [scikit-learn PLS](#) and [Prior Labs’ TabPFN](#). Architecture citations and source mappings are collected in `MODEL_REFERENCES.md`.

4 Results and interpretation

Compare RMSE within each task, in the original target units. For an overall score, rank the $M = 15$ models by test RMSE on each of the $D = 30$ splits, using average ranks for ties, and compute $\text{Score}_m = \frac{100}{D} \sum_{d=1}^D \frac{M-r_{md}}{M-1}$. Each task has equal weight; 100 means first place everywhere. The score measures rank consistency rather than error reduction. Compare scores only for identical model and task sets.

Model	Score	Mean rank	Mean R^2	Wins	Time (s)
TabPFN-3.5	94.52	1.77	0.6909	12	4.99
PLS regression	88.33	2.63	0.6994	16	14.16
IPA	75.95	4.37	0.5455	1	49.66
DeepSpectra	68.33	5.43	0.5401	1	77.96
1D-CNN_1L_3	67.38	5.57	0.4997	0	37.80
ResidualSpectra	65.48	5.83	0.5560	0	33.40
SpectraTr	44.52	8.77	0.3437	0	128.85
1D-CNN_1N_3	44.29	8.80	0.2159	0	38.02
SpectraNet32	42.86	9.00	0.0784	0	30.39
1D-CNN_3_1	37.86	9.70	0.0857	0	49.58
MarkSpectra	30.24	10.77	0.2520	0	28.74
Spectraformer	29.76	10.83	0.2982	0	70.69
1D Inception-ResNet	25.00	11.50	-0.1644	0	90.36
SpectraNet53	24.52	11.57	0.0366	0	49.90
SCNet	10.95	13.47	-0.7680	0	18.22

Table 3: Results for all 15 models across 30 tasks. Score is the mean normalized RMSE rank; wins count the lowest test RMSE. Time is averaged over tasks: one final fit and prediction for DL and TabPFN-3.5, or the complete CV search for PLS.

The overall score is led by TabPFN-3.5 (94.52), followed by PLS regression (88.33) and IPA (75.95). Lowest-RMSE task counts are TabPFN-3.5: 12; PLS regression: 16; IPA: 1; DeepSpectra: 1.

Runtime and PLS selection. Run times presented here are based on an Intel Core i9-13900K CPU and NVIDIA RTX 2080 Ti GPU (11 GB), with Windows 11/WSL2 and TensorFlow 2.20 for DL. Times are comparative on this hardware, with different workloads: DL averages construction, compilation, final fit and train/test prediction per seed, excluding CV; TabPFN-3.5 measures fit and test prediction; PLS measures the entire CV search, excluding final fit and prediction.

PLS selects transformed spectra on 23 tasks. Its search averages 14.16 seconds per task (424.90 seconds total); Figure 1 plots the selections and search time against training sample count.

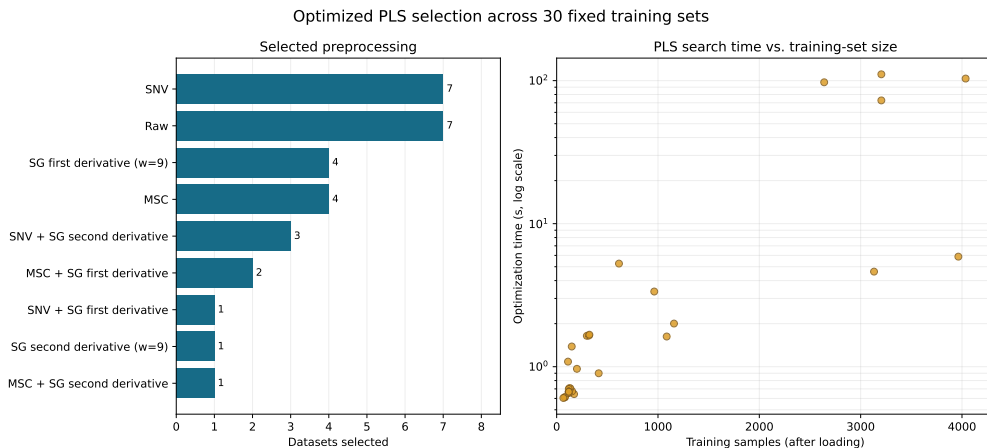


Figure 1: PLS preprocessing selections (left) and optimization time versus training sample count after loading (right; one point per task, logarithmic time axis). Spectral widths also differ, so this is an observed relationship across datasets, not a controlled sample-size scaling experiment.

4.1 Individual tasks and practical limits

Table 4 compares the two references with the best DL result on each task. The appendix gives all 15 models’ test RMSE values. The best-DL column is a retrospective summary, not a model-selection procedure: choosing an architecture using these test scores would no longer be an independent test.

Table 4: Held-out test results for the 30 tasks in Table 1. RMSE is in each target’s original units. The standard deviation is across ten TabPFN-3.5 seeds on the same split. The last two columns give the lowest mean RMSE among the 13 DL models and its model. Bold marks the lowest mean RMSE among TabPFN-3.5, optimized PLS, and all DL models.

Task	TabPFN-3.5 RMSE	Run SD	TabPFN-3.5 R^2	PLS RMSE	Best DL RMSE	Best DL model
1	1.1802	0.0191	0.5402	0.6292	1.1832	1D-CNN_1L_3
2	0.4107	0.0427	0.9758	0.1710	0.2895	IPA
3	0.8204	0.0233	0.9933	1.9486	1.2416	IPA
4	0.4648	0.0344	0.9992	0.3263	1.5149	1D-CNN_1L_3
5	0.3604	0.0048	0.9622	0.4318	0.3568	IPA
6	0.4891	0.0073	0.1423	0.4804	0.5079	DeepSpectra
7	0.3426	0.0046	0.2410	0.3876	0.3847	SpectraNet53
8	0.5840	0.0148	0.7308	0.9470	0.6828	DeepSpectra
9	0.3408	0.0071	0.5372	0.4125	0.3573	ResidualSpectra
10	0.4923	0.0125	0.1700	0.5248	0.4813	DeepSpectra
11	0.5123	0.0109	0.9558	0.5142	0.5641	1D-CNN_1L_3
12	2.0486	0.0158	0.5905	2.0405	2.2317	ResidualSpectra
13	0.3106	0.0100	0.2501	0.0586	0.1410	DeepSpectra
14	2.3985	0.0544	-0.0307	2.6872	3.0899	Spectraformer
15	1.0116	0.0079	0.3951	0.8188	0.9340	DeepSpectra
16	1.0504	0.0096	0.6343	0.9535	1.1174	DeepSpectra
17	0.6628	0.0420	0.9581	0.4474	0.7341	ResidualSpectra
18	0.9480	0.0288	0.6600	0.8232	1.3996	SpectraNet32
19	0.8326	0.0061	0.6369	0.8963	0.8353	DeepSpectra
20	0.5590	0.0162	0.8938	0.4239	0.5616	ResidualSpectra
21	0.8618	0.0061	0.5610	0.9677	0.8918	DeepSpectra
22	0.7908	0.0073	0.9080	0.9889	0.9154	SpectraNet32
23	0.6379	0.0109	0.9102	0.7475	0.7162	IPA
24	0.6333	0.0124	0.9233	0.6944	0.6627	SpectraNet32
25	0.7298	0.0158	0.9268	0.8293	0.7829	IPA
26	1.1702	0.0616	0.9456	0.6243	1.0950	IPA
27	0.9453	0.0142	0.4679	0.7572	0.9246	ResidualSpectra
28	0.3339	0.0113	0.9809	0.3311	0.3568	IPA
29	0.5565	0.0240	0.9281	0.2616	0.5996	IPA
30	0.3539	0.0074	0.9379	0.2934	0.3694	1D-CNN_1L_3

The reported standard deviations describe variation across seeds on a fixed split. They are not confidence intervals for performance on new samples. Several tasks share source samples, instruments, or targets; the collection comprises 30 task configurations rather than 30 independent studies.

Large differences between architectures can reflect optimization, preprocessing, regularization, or implementation choices as well as network structure. Version 0.1 establishes a reproducible starting point; it does not establish that any architecture is universally best. For a new application, inspect the task-level results and validate on an independent sample collection.

5 Run the benchmark

5.1 Install and check the data

Obtain the [public repository](#) and run commands from its root directory. The reference environment uses Python 3.13. Create the supplied Conda environment and validate the checkout:

```
git clone https://github.com/DeepLightLab/NIRBENCH_DL_v0.1.git
cd NIRBENCH_DL_v0.1
conda env create -f environment.yml
conda activate nirbench-dl-v01
python validate_setup.py
```

The validator checks the 30 tasks, 60 stored split files, shapes, finite values, and file hashes. For a virtual environment instead, install `requirements.txt`; see the repository README for platform-specific TensorFlow and PyTorch setup. A compatible NVIDIA GPU is recommended for the neural models.

TabPFN-3.5 weights are distributed separately from the code. Obtain the standard v3.5 checkpoint following the [official TabPFN setup instructions](#) and applicable model terms. The launcher uses the TabPFN cache or an explicit `-model-path`; installing the Python package alone does not provide the checkpoint.

5.2 Run all methods or a subset

```
# All 13 DL architectures and all 30 tasks
python benchmark.py --model all

# One architecture, optionally restricted to one task
python benchmark.py --model ipa
python benchmark.py --model ipa --dataset 1-Wheat_kernels_protein

# Reference methods on all tasks
python pls_baseline.py
python benchmark_tabPFN.py --device cuda
```

Both reference launchers also accept `-dataset`. For an explicit checkpoint location, add `-model-path /path/to/tabPFN-v3.5-20260909.safetensors`. Use `-help` on any launcher to see its options. Keep the default folds, seed count, preprocessing search, and training settings when reproducing this report.

To resume a DL run, set the environment variable `NIRBENCH_RESUME=1`; completed task outputs are skipped. TabPFN-3.5 automatically skips complete outputs with the same recorded protocol; `-overwrite` reruns them. Resume only when the code and configuration match the existing results.

5.3 Read the outputs

DL and TabPFN-3.5 write to `results/<model>/`. The `<model>_summary.csv` and `<dataset>_metrics.csv` files contain the mean and sample standard deviation of RMSE and R^2 , plus average runtime. Predictions and plots represent the last seeded run, whereas metrics aggregate all ten runs. DL also saves a training-loss plot; TabPFN-3.5 saves per-seed metrics and a protocol JSON file.

PLS writes the selected preprocessing, latent variables, CV/test metrics and search time to `pls_baseline_benchmark.csv`, and candidate CV scores to `pls_preprocessing_search.csv`. The `.csv` extension is intentional. Keep these result files, the code revision, package versions and run commands together. The DL runner's `-plot-only` option summarizes available model result folders; its ranking depends on which results are present and is not automatically the complete 15-model comparison in this report.

6 Implement and benchmark a new model

6.1 Provide a Keras model builder

Copy a Python file into `models/newmodel/` in the public repository. Define a function that accepts the spectral feature count and returns a fresh, uncompiled Keras model. The input must have shape `(features, 1)` per sample, and the output should be a single linear regression value. Other builder arguments must have defaults. For example:

```
# models/newmodel/MyModel.py
from tensorflow import keras
from tensorflow.keras import layers

def build_model(input_vector_dimension):
    inputs = keras.Input(shape=(input_vector_dimension, 1))
    x = layers.Conv1D(16, 9, padding="same", activation="relu")(inputs)
    x = layers.GlobalAveragePooling1D()(x)
    outputs = layers.Dense(1)(x)
    return keras.Model(inputs, outputs)
```

Let the benchmark handle data loading, scaling, compilation, training, and scoring. The builder should work across different spectral widths and return newly initialized weights every time it is called. Avoid training at module import or reading benchmark labels from the model file.

6.2 Test one task, then run the full collection

```
python benchmark_newmodel.py --model-file MyModel.py --dataset 1-Wheat_kernels_protein
python benchmark_newmodel.py --model-file MyModel.py
```

If there is only one Python model file in the folder, `python benchmark_newmodel.py` selects it automatically. Use `-builder FUNCTION` for a different factory name, `-name MyModel` for a distinct output label, and `-list` to list available files. Choose a new name when changing the architecture or training configuration so that saved runs remain distinguishable.

The external-model launcher reuses the shared benchmark functions: feature/target standardization, Adam at 10^{-3} , MSE loss, batch size 16, five-fold epoch selection (up to 600 epochs), and ten final seeds. It writes the same metrics, predictions, plots, and summary schema under `results/MyModel/`, then updates the ranking of available result folders. `-batch-size` changes the batch size; custom optimizers or schedules require an explicit extension of the training code and should be documented as a different configuration.

6.3 Make the comparison reproducible

Develop and tune the model using training data and internal validation only. Once the design is fixed, run all 30 tasks and report the per-task test metrics with their seed variability. Recompute aggregate ranks using the same competitors and shared tasks; a score from a partial run is not comparable to the 30-task score here.

Share the model source, builder settings, dependency versions, code revision, commands, and complete result files. If repeated inspection of the public test scores influences further design choices, treat those splits as development feedback and use additional held-out data for a confirmatory claim.

7 Dataset origins and attribution

The 30 tasks are curated from the sources below, following the repository’s [DATASETS.md](#). Task numbers match Table 1 and the dataset-folder prefixes. Multiple targets, tomato groups, mango seasons, and the transformed pear configuration produce separate tasks from shared source collections. Benchmark files are harmonized copies with fixed train/test partitions; original splits were retained when supplied.

Tasks	Material and target	Origin and source reference
1	Wheat kernels; protein	University of Copenhagen chemometrics datasets; Nielsen et al. (2003). Dataset collection ; doi:10.1094/CCEM.2003.80.3.274.
2	Wheat flours; protein	D. Bertrand / INRA; distributed through ChemProject ChemData .
3	Tecator meat; moisture	Karin Thente / Tecator AB; StatLib Tecator dataset .
4	Grain; glucose	T. Næs and T. Isaksson; Eigenvector research datasets .
5	Cucurbitaceae fruit; soluble solids	Kusumiyati et al.; Mendeley Data, version 2. doi:10.17632/k55b8mvs84.2.
6–10	Five tomato groups; soluble solids	Ibañez et al.; Zenodo. The source is divided into five tomato-group tasks. doi:10.5281/zenodo.10633732.
11	Olive oils; C16:0 fatty acid	N. Dupuy’s group, University of Aix-Marseille; distributed through ChemProject ChemData .
12	Diesel fuels; cetane number	Scott Hutzler, Southwest Research Institute; Eigenvector SWRI dataset .
13	Milk; protein	Díaz-Olivares et al.; Zenodo. doi:10.5281/zenodo.8263430.
14	Pharmaceutical tablets; assay	IDRC 2002 shootout, using spectrometer 1; Eigenvector research datasets .
15, 21	Rocha pear 2021; soluble solids	CEOT, University of Algarve. Task 21 uses trimmed, SNV-transformed spectra from the same source as task 15. Original measurements available from the author. doi:10.1016/j.postharvbio.2021.111562.
16	Rocha pear 2019; soluble solids	CEOT, University of Algarve; Passos et al. (2019). Original measurements available from the author. doi:10.3390/s19235165.
17–18, 26–29	Barley (17–18), corn (26–27), wheat (28–29); moisture and protein	sensAIfood / M. Lagerholm; Perten instruments; Zenodo. Each cereal yields moisture and protein tasks. doi:10.5281/zenodo.15838136.
19–20	Wheat; moisture and protein	sensAIfood / P. Berzaghi; Grainit/AuroraNIR; Zenodo. doi:10.5281/zenodo.15838272.
22–25	Mango seasons S1–S4; dry matter	Anderson et al.; original <code>NAnderson2020MendeleyMangoNIRData.csv</code> (V2). The benchmark separates four harvest seasons. doi:10.17632/46htwnp833.
30	Wheat; protein	sensAIfood / CRA-W; FOSS NIRSYSTEM-5000; Zenodo. doi:10.5281/zenodo.16108496.

Cite the original dataset records and associated papers when using these tasks. The benchmark’s software license does not replace the attribution requirements or terms attached to the source datasets. For CEOT-UAlg measurements, contact the repository author for provenance and reuse information.

Appendix: complete test RMSE matrix

Task	PLS	TabPFN-3.5	DeepSpectra	IPA	Inception-ResNet	MarkSpectra	ResidualSpectra	SCNet	Spectraformer	SpectraTr	SpectraNet32	SpectraNet53	CNN 1L_3	CNN 1N_3	CNN 3_1
1	0.6292	1.1802	1.3566	1.3113	1.3688	1.3474	1.3396	1.3321	1.3781	1.3104	1.3338	1.3280	1.1832	1.3130	1.3200
2	0.1710	0.4107	0.6347	0.2895	1.8485	1.7948	0.4769	2.5169	1.6628	1.6201	1.8410	1.9312	0.7119	1.2286	1.4392
3	1.9486	0.8204	2.2222	1.2416	3.7196	5.9166	1.4609	8.4091	6.5707	1.6415	3.9409	6.8195	1.5228	3.5799	1.7922
4	0.3263	0.4648	3.3473	1.6557	2.8783	4.2620	2.1756	3.9953	6.5136	2.2081	3.3316	5.1308	1.5149	1.8081	2.5270
5	0.4318	0.3604	0.4930	0.3568	0.5624	0.4767	0.4625	0.5995	0.6007	0.5039	0.4240	0.4623	0.5104	0.5098	0.4144
6	0.4804	0.4891	0.5079	0.5391	0.6635	0.5349	0.5103	0.5538	0.5374	0.5342	0.5345	0.5566	0.5255	0.5412	0.5442
7	0.3876	0.3426	0.4372	0.3980	0.3994	0.4041	0.4133	0.5269	0.3877	0.4777	0.3976	0.3847	0.4261	0.4301	0.4018
8	0.9470	0.5840	0.6828	0.6965	1.2328	1.0380	0.7646	1.1247	1.0262	1.0389	1.1525	1.0975	0.7562	0.8488	1.0485
9	0.4125	0.3408	0.4025	0.3832	0.5352	0.5542	0.3573	0.4872	0.4604	0.4767	0.4183	0.5552	0.4078	0.4437	0.4745
10	0.5248	0.4923	0.4813	0.5321	0.6199	0.5654	0.5745	0.5753	0.6131	0.5900	0.5528	0.5870	0.5311	0.5701	0.5816
11	0.5142	0.5123	0.6589	0.5935	2.8290	1.4196	0.5698	2.2930	1.4698	0.9242	1.2969	2.2312	0.5641	0.7050	0.9729
12	2.0405	2.0486	2.4279	2.2532	4.2835	2.6709	2.2317	2.8709	2.7165	2.6341	2.4329	2.8114	2.6981	2.5361	2.4779
13	0.0586	0.3106	0.1410	0.3171	0.3769	0.3642	0.3414	0.3361	0.3449	0.3325	0.3465	0.3431	0.3265	0.3567	0.3430
14	2.6872	2.3985	4.3192	3.1985	6.8450	3.7460	3.1307	11.7607	3.0899	3.8212	7.4201	6.4953	3.9895	6.2930	7.2377
15	0.8188	1.0116	0.9340	1.1588	1.2373	1.1540	1.3040	1.3576	1.2474	1.2358	1.2518	1.3028	1.0567	1.3243	1.2249
16	0.9535	1.0504	1.1174	1.2342	1.6191	1.3730	1.4404	1.5453	1.4508	1.5168	1.4809	1.5129	1.2838	1.4248	1.5260
17	0.4474	0.6628	0.9762	0.7651	1.8782	3.0747	0.7341	3.4431	2.3852	1.8235	3.2329	3.0537	1.0917	1.6853	2.9458
18	0.8232	0.9480	1.5112	2.1844	2.1881	1.5592	1.4860	1.6478	1.4665	1.8499	1.3996	1.4777	1.6553	1.8798	2.0656
19	0.8963	0.8326	0.8353	0.8447	1.1160	1.2861	0.9118	1.2827	1.2648	0.9851	1.2218	1.2390	0.8539	1.0110	1.2947
20	0.4239	0.5590	0.7888	0.8136	1.2769	1.3654	0.5616	1.4143	1.2107	1.0611	1.3034	1.3482	0.6482	1.1513	1.0832
21	0.9677	0.8618	0.8918	0.9021	1.0673	1.2273	1.0367	1.2901	1.2265	1.0823	1.0840	1.2576	0.9615	1.0290	1.2518
22	0.9889	0.7908	0.9880	0.9545	1.0435	1.5707	1.0589	1.9229	1.7480	0.9842	0.9154	1.2613	0.9871	1.1784	1.3074
23	0.7475	0.6379	0.7415	0.7162	0.8920	1.3530	0.8304	1.7098	1.5821	0.9777	0.8222	1.4001	0.7882	1.0901	0.9981
24	0.6944	0.6333	0.7099	0.6900	0.9432	1.5573	0.7160	2.0526	1.5238	0.7495	0.6627	0.8553	0.6945	0.8711	0.9497
25	0.8293	0.7298	0.8421	0.7829	0.8962	1.5014	0.8589	2.2113	1.7297	1.0095	0.9894	1.5230	0.8846	1.2136	1.3487
26	0.6243	1.1702	1.4397	1.0950	3.8197	4.0571	1.8445	5.2042	4.3250	4.4427	4.6215	4.9443	1.6667	3.5900	2.9262
27	0.7572	0.9453	1.2553	1.1419	1.7059	1.3584	0.9246	1.5053	1.4389	1.5092	1.4928	1.6910	1.1923	1.1530	1.3891
28	0.3311	0.3339	0.6420	0.3568	1.0007	1.8009	0.4665	2.8553	1.8776	1.3826	2.0220	2.2090	0.6084	1.1085	0.9433
29	0.2616	0.5565	0.6957	0.5996	2.0289	1.8938	0.7605	1.8115	1.6837	1.6635	1.7856	1.7001	0.8742	1.2966	1.2198
30	0.2934	0.3539	0.4409	0.6522	0.4661	0.8780	0.3718	1.2393	0.6620	0.4975	0.4109	0.8307	0.3694	0.5114	0.9439

Table 6: Test RMSE for all 15 models in the original target units. Bold marks the minimum in each row, based on unrounded values. Neural-network and TabPFN-3.5 entries are means across ten seeds; PLS is one final fit. Task names and sample counts are in Table 1. The three CNN columns denote the simple benchmark baselines.